

Sumário do Resultado: Design e Implementação do Software de Reserva de Salas

Autor: Cleydson Soares de Freitas

Componente Curricular: DevSecOps e Ciclo de Vida de Software

Data: 14 de Maio de 2026

1. Visão Geral e Alinhamento com o SDLC

Este relatório apresenta o sumário do resultado técnico focado no **Passo 4 (Design e Implementação)** do Ciclo de Vida de Desenvolvimento de Software (SDLC). As etapas preliminares de planejamento, análise de papéis e levantamento de requisitos foram executadas para fundamentar as regras de negócio aqui descritas. O sistema foi projetado sob a ótica de **DevSecOps**, integrando validações de segurança da informação diretamente no núcleo lógico da aplicação desde a fase de design.

2. Requisitos de Sistema e Controle de Acesso (RBAC)

O software opera sob o modelo de Controle de Acesso Baseado em Papéis (RBAC). As permissões do sistema tratam rigorosamente as premissas estabelecidas no escopo do projeto:

- **Usuário Comum (Privilegio Padrão):** Autenticação no sistema, listagem de salas e horários disponíveis, e requisição de novas reservas de salas.
- **Usuário Administrador (Privilegio Admin):** Acesso total às funcionalidades do sistema, possuindo controle exclusivo sobre o expurgo e cancelamento de reservas.

3. Esboço da Interface do Usuário (Mockup CLI)

Para validar a experiência do usuário (UX) e garantir a portabilidade da aplicação, foi desenvolvido um protótipo conceitual baseado em Interface de Linha de Comando (CLI) estruturada:

Plaintext

=====

SISTEMA DE GESTÃO DE RESERVAS DE SALAS

=====

Usuário Autenticado: Cleydson (Nível de Acesso: ADMIN)

=====

[1] Efetuar Nova Reserva de Sala

[2] Listar Todas as Reservas Ativas

[3] Cancelar Reserva Existente (Restrito a Administradores)

[4] Alternar Perfil de Usuário / Efetuar Logout

[5] Encerrar Aplicação

=====

>> Selecione a opção desejada: _

4. Pseudocódigo e Lógica de Implementação

A lógica central da aplicação foi projetada em pseudocódigo estruturado para garantir o tratamento robusto contra duas falhas críticas identificadas no levantamento de requisitos: a colisão/conflito de horários e a elevação não autorizada de privilégios.

Início Algoritmo_Reserva_Salas

Procedimento EfetuarReserva(sala_solicitada, horario_solicitado, usuario_ativo)

Para cada registro em Lista_Geral_De_Reservas Faça

Se (registro.sala == sala_solicitada) E (registro.horario == horario_solicitado) Então

Escreva("[ERRO] Conflito Detectado: Sala já ocupada neste horário.")

Retornar

FimSe

FimPara

Criar_Nova_Reserva(sala_solicitada, horario_solicitado, usuario_ativo)

Escreva("[SUCESSO] Reserva confirmada e armazenada.")

FimProcedimento

Procedimento CancelarReserva(id_da_reserva, usuario_ativo)

Se (usuario_ativo.isAdmin == Falso) Então

Escreva("[ACESSO NEGADO] Operação estrita a perfis administrativos.")

Gerar_Log_De_Auditoria("Tentativa de bypass por: " + usuario_ativo.nome)

Retornar

Senão

Expurgar_Reserva_Por_ID(id_da_reserva)

Escreva("[SUCESSO] Registro removido do sistema.")

FimSe

FimProcedimento

Fim Algoritmo_Reserva_Salas

5. Estratégia de Arquitetura Técnica e Próximas Etapas

A implementação do protótipo local foi estruturada na linguagem **Java**, dividida em pacotes lógicos claros (Modelos para encapsulamento de entidades e a classe Principal para o controle de fluxo).

Como parte do pipeline de melhorias contínuas previstas pelo ciclo do SDLC:

1. **Persistência de Dados:** Transição das estruturas de dados em memória (ArrayList) para um banco de dados relacional (SQLite), assegurando atomicidade e consistência.
2. **Auditoria DevSecOps:** Submissão do código-fonte a ferramentas de análise estática (SAST) para mitigar possíveis vulnerabilidades de controle de acesso impróprio ou estouro de parâmetros.